MODDON 14.03.2025

Exercise A: For the following tasks use the table **EMPLOYEES**

1. Display the first name, the last name and the salary for Abel.
2. Arrange all the employees using salary and display the first_name, the last_name and the salary.
3. Decrease the salary by 200 and display the first name, the last name, the old salary and the new salary for employees having department id=90.
4. Display the average salary for employees where department_id = 90.
5. Display the average salary for employees where job_id = it_prog and round to two decimal places.
6. Display the latest hire date for department_id=90.
7. Display the last name in the alphabetic list of names.
8. Display the first in the alphabetic list of names name for employees having department_id = 90.
9. Display the lowest salary from employees having manager_id= 100.
10. Display all the information about employees having last_name starting with „K”.
11. Display first_name and last_name in a new column “Name” from emoployees having job_id = sa_rep.
12. Display the last_name with capital letters for employees having department_id= 50 or 80.
13. Display the first_name, the last_name, and the salary for employees who earn more than Ernst. Order by last_name.
14. Display all the information for employees who work in the same department as Davies.
15. Display all the information for employees who earn less that Lorentz and are in the same department as Davies.
16. Display the first name, the last name and the hire date for employees who have the same job_id as Rajs.
17. Display the first name, the last name and the hire date for employees hired after Davies.
18. Display the first name, the last name and the hire date for employees who have the same job id as Rajs and were hired after De Haan.

19. Arrange all the employees using hire_date, and display first_name, last_name and the location_id from employees who earn more than Davies.
20. Display all the information for Davies' manager.
21. Display the first_name, the last_name, and the salary for Davies' manager.
22. Arrange all the employees having the same department as Matos using salary, and display first_name, last_name and the salary.
23. Display the first_name, the last_name, and the salary for employees who earns more than the minimum salaries for all the employees having department_id=80.
24. Arrange all the employees having the same department as Abel using salary, and display first_name, last_name and the salary.
25. Arrange all the employees hired after Abel using salary, and display first_name, last_name, salary and the hire_date.

Exercise B: For the following tasks use the tables: **EMPLOYEES**, **DEPARTMENTS, LOCATIONS, JOB_HISTORY.**

26. Display the first_name, the last_name, the salary and the hire_date (from employees) and the department_name and the location_id(from departments).
27. Display the first_name, the last_name, the salary, the department_id and the hire_date (from employees) and the department_name and the location_id(from departments) for employees having salary>8000.
28. Display the first_name, the last_name, the salary and the hire_date (from employees) and department_name and location_id(from departments) for employees who earn more than Ernst. Order by last_name.
29. Display the first_name, the last_name, the salary(from employees), the department_name(from departments) the street_address and the city(from locations).
30. Display the first_name, the last_name, the salary(from employees), the department_name(from departments) the street_address and the city(from locations) for employees who earn more than Grant. Order by salary.
31. Display the salary, bonus, first_name and last_name(from employees) and department_name and manager_id(from departments)
32. Display the salary, bonus, first_name and last_name(from employees) and department_name and manager_id(from departments) where manager_id=124.

33. Display the salary, bonus, first_name and last_name(from employees) and department_name and manager_id(from departments) where salary> 10000.
34. Display first_name and last_name(from employees) and location_id(from departments) from employees hired after 01.09.1990.
35. Display the first_name, the last_name, the salary(from employees) and department_name and location_id(from departments) for employees who earn less that the average of the salaries for employees having department_id =50 or 90.
36. Display the first_name, the last_name and the salary(from employees), the department_name, and the location_id(from departments) from employees having the same department_name as Matos.
37. Increase the salary with 500 and display the first_name, the last_name, the salary, the new salary(from employees) and the location_id(from departments) for employees hired after Davies.
38. Create a join between the tables EMPLOYEES and DEPARTMENTS and display all the information from both tables for employees having department_id=90.
39. Create a natural join between the tables EMPLOYEES and JOB_HISTORY and display all the information from both tables.
40. Create a join between the tables EMPLOYEES and JOB_HISTORY using department_id and display the first_name, last_name and salary from employees and start_date and end_date from job_history.
41. Create a join between the tables EMPLOYEES and JOB_HISTORY using department_id and display the first_name, last_name and salary from employees and start_date and end_date from job_history for employees having first_name starting with „M".
42. Create a join between DEPARTMENTS and LOCATIONS and display the department_id, the department_name, the street_address, the city, and the postal_code for the employees having the same department_id as Matos.
43. Create a join between EMPLOYEES, DEPARTMENTS and LOCATIONS and display the first_name, the last_name, the salary, the manager_id, the department_name, the location_id, the street_address, and the city for all the employees having the same manager as Rajs.
44. Create a join between DEPARTMENTS and EMPLOYEES and display the first_name, the last_name, the salary, the department_name and the location_id

for all the employees having the same manager as Matos. Arrange in descending order using the salary.

45. Create a cross join between the tables EMPLOYEES and JOB_HISTORY and display all the information from both tables.
46. Create a natural join between the tables EMPLOYEES and JOB_HISTORY and display the first_name, last_name, the department_id, the start_date and the end_date.
47. Create a natural join between the tables EMPLOYEES and JOB_HISTORY and display the first_name, last_name, the department_id, the start_date and the end_date.
48. Create a natural join between the tables EMPLOYEES and JOB_HISTORY and display the first_name, last_name from employees and start_date and end_date from job_history.
49. Create a join between the tables EMPLOYEES and JOB_HISTORY using employee_id and display the first_name, last_name, the employee_id, the start_date and the end_date.
50. Create a cross join between the tables DEPARTMENTS and JOB_HISTORY and display all the information from both tables.
51. Create a natural join between the tables DEPARTMENTS and JOB_HISTORY and display the department_name, location_id from departments and start_date and end_date from job_history.
52. Create a join between the tables DEPARTMENTS and JOB_HISTORY using department_id and display the department_name, location_id, the start_date and the end_date.
53. Create anatural join between the tables EMPLOYEES and DEPARTMENTS and display all the information from both tables for employees having department_id=80.
54. Make a join between EMPLOYEES and DEPARTMENTS and display first_name, last_name, salary, hire_date, department_name and manager_id.
55. Create a join between EMPLOYEES and DEPARTMENTS and display all the information about employees who work in the same department as Matos. Arrange all the employees using the hire_date.

56. Create a join between the tables EMPLOYEES and JOB_HISTORY and display the first_name, last_name, salary, start_date, end_date from employees having department_id=50.

Exercise C: For the following tasks use the tables: **REGIONS** and **COUNTRIES**

57. Create a cross join between the tables REGIONS and COUNTRIES and display all the information.
58. Create a join between the tables REGIONS and COUNTRIES using region_id and display all the information.
59. Create a join between the tables REGIONS and COUNTRIES using region_id and display all the information about the Canada.
60. Create a join between the tables REGIONS and COUNTRIES using region_id and display all the information about the countries having region_id=2.